

OpenACC and the pgcc/nvc compiler

COMP 445

Standards

We have seen two *standards* so far:

- OpenMP

- MPI

These standards are developed to guide compiler writers

They define the syntax for the pragmas (in the case of OpenMP)

They define the functions that should be present in the library, e.g.

- `omp_get_num_threads()`

- `MPI_Send(...)`

More than one compiler can implement the standard (including fortran language!)

- gcc

- clang

- (there are others also)

- mpich

- openMPI

Another standard for GPU and multicore CPU

OpenACC (openacc.org)

The ACC is for accelerator (a common term for a separate card)

Defines pragmas and a handful of functions

Some pragmas are the same as OpenMP:

- private

- reduction

Many others are related to the accelerator device needs

- for specifying blocks (called gangs)

- for moving data

- advising the compiler that a loop is 'independent'

See <https://www.openacc.org/sites/default/files/inline-files/OpenACC.2.6.final.pdf> for an example version of the OpenACC specification.

The pgcc/nvc compiler

Implements the OpenACC standard

nvc is NVIDIA's new name for pgcc

Is being maintained by NVIDIA

Designed to work on both NVIDIA cards and AMD GPU cards

Philosophy: same code, build to different architectures,
both CPU multicore (including OpenMP) and GPU

Takes a lot of experiments and code writing off your hands for:
choosing CUDA block sizes
defining the data movement

The activities

Designed to take you on an incremental journey

1. gcc and its compiler optimization levels
2. openMP with both gcc and pgcc
3. Fast multicore version with pgcc
4. GPU version with pgcc

Two examples in separate repos

Basics, then Vector addition (very simple)

Matrix Multiply